SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I Sharmuganathan 192511347 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Sign
Signature of the student:

Scanned with OKEN Scanner

Q1) Self Driving Car Analysis

* Percepts: video streams, LIDAR Points Clouds, radar data, GPS Coordinates, odometer reading, ultrasonic sensor data, and accelerometer data

* Actions: Steer Steering Wheel, accelerate, decelerate, activate turns Signals, and Sound horn

2) Environment Characterization:-

* partially observable:-

The car cannot see through buildings, around blind corners or know the internal intention of nearby drivers

* Stochastic:-

Traffic pattern, pedestrian actions, and mechanical failures cannot be predicted with absolute 100% certainty

* Dynamic:-

The environment constantly changes while the agent is deciding on its next action

* Continuous:-

Dimension such as absolute position speed, steering angle, and elapsed time vary smoothly over continuous numerical domains rather than fixed distinct steps.

Scanned with OKEN Scanner

3) Most Suitable agent Architecture :-

* Utility-Based Agent Architecture is the optimal choice
* Justification :- A Self driving vehicle must simultaneously handle multiple conflicting trade offs such as minimizing travel duration, ensuring passenger safety, maintaining structural comfort and adhering to strict illegal regulations. A Utility function numerical weight these trade offs to evaluates which route or maneuver maximizes safety and comfort, whereas goal based-agents only look for any valid path to the destination without optimizing performance quality.

Q2 Online Search agent for cleaning Robot.

1) Exploration Strategy :-

There is no pre existing map, the online Search agent alternates between Calculating a Step physically executing the move and gathering local sensor percepts from that new position.

* Dynamic Graph Construction

The robot dynamically constructs a topological map or grid representation of its environment.

Scanned with OKEN Scanner

Exploration Policy :- nodes, edges and frontiers

It uses an online search algorithm - such as online Depth First Search (DFS) or learning real time A*(LRTA*) to navigate towards unvisited frontiers and backtracks safely when detecting an obstacle or dead end.

2) Challenges of online search vs offline search :-

* Irreversibility of Actions :-

Offline search executes safe mental simulation inside a complete memory model where paths can be discarded. Online search involves actual physical action. Physical actions that can lead to irreversible damage or dead ends.

* Suboptimality :-

Without a global view of the architecture beforehand, the agent cannot compute the absolute shortest path and must rely on localized decisions.

* Competitive Ratio :-

The exploration path taken online is always sub optimal compared to a calculated offline shortcut due to unavoidable trial and error discovery

Q3 Bidirectional Search Vs Breath-First Search

* Breath-First Search (BFS):

Explores uniformly outwards from the Start nodes s layers by layer until it reaches the goal G. Time Complexity where b is the branching factors and d is the depth of the Solution.

* Bidirectional Search:-

Runs two concurrent BFS operationals - one forward starting from S and one backwords starting from G. The Search terminates when two frontiers intersect. Time Complexity is $O(b^{d/2} + b^{d/2}) = O(b^{d/2})$

Illustrative Example:-

Consider a graph with branching factor of b=3 where the shortest path from S to G lies at depth d

* Using Standards BFs

Nodes Explored = $3^1 + 3^2 + 3^3 + 3^4 = 3 + 9 + 27 + 81 = 120$

* Using Bidirectional Search

The two frontiers will meet halfways at depth $d/2 = 2$

Scanned with OKEN Scanner

Forward Search Explores $3^1 + 3^2 = 12$ nodes

Backward Search Explores $3^1 + 3^2 = 12$ nodes

Total nodes Explored = $12 + 12 = 24$ nodes

By splitting the exponential growth in half, the bidirectional search explores only 24 nodes instead of 120, reducing the search space dramatically even in small graphs.